

Python

Generators, one value at a time

How **yield** turns a function into a lazy stream — and lets you process data that would never fit in memory. Eight runnable examples, no dependencies.



Alejandro Sánchez Yalí

Software Developer · AI & Blockchain

1 What a generator is

A function with a `yield` in it doesn't return a value — it returns a **generator**: an iterator that produces values lazily, one at a time, only when asked. It never builds the whole sequence.

```
basic.py

def count_up(n):
    i = 0
    while i < n:
        yield i
        i += 1

for x in count_up(3):    # 0, 1, 2
    ...
```

Calling `count_up(3)` runs **no code** yet; it hands back a generator. The body only advances when the `for` loop pulls the next value.

WHY IT MATTERS

This is how you handle streams and huge (or infinite) data: you hold **one value at a time**, not the whole list. A log reader, a paginated API, a number sequence, all in constant memory.

2 yield pauses and resumes

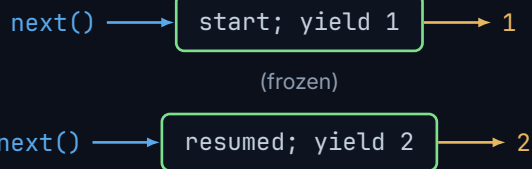
Each `yield` hands back a value and **freezes** the function, keeping its locals and its exact position. The next pull resumes right after that `yield`, state intact.



● ● ● pause.py

```
def gen():
    print("start")
    yield 1
    print("resumed")
    yield 2

g = gen()
next(g)  # prints "start", returns 1
next(g)  # prints "resumed", returns 2
```



Each `next()` resumes the frozen function, runs to the next `yield`, hands back a value, and freezes again.

GOTCHA

A generator is **exhausted after one pass**. Loop over it twice and the second loop sees nothing; the state ran to the end. Rebuild it (or materialise a `list`) when you need to iterate again.

3 Generator expressions save memory

Swap the brackets of a list comprehension for parentheses and you get a lazy generator instead of a materialised list. Same syntax, wildly different memory.



genexpr.py

```
nums = [x * x for x in range(10_000_000)] # ~400 MB list in RAM
nums = (x * x for x in range(10_000_000)) # a generator, a few
    hundred bytes

total = sum(x * x for x in range(10_000_000)) # streamed, no list
    built
```

GOTCHA

The expression is **lazy**: nothing runs when you write it. Side effects and exceptions are deferred to the moment you consume it, not the moment you create it.

4 Chaining generators into a pipeline

Because each stage is lazy, you can chain generators and a **single item** flows all the way through per pull. The memory stays flat no matter how big the input.

pipeline.py

```
from itertools import islice

lines = (line.rstrip() for line in open("app.log"))
errors = (l for l in lines if "ERROR" in l)
first = islice(errors, 10) # nothing has been read yet
for e in first:
    print(e) # now it pulls, one line at a
    time
```





The consumer pulls; one line travels the whole chain at a time, so memory stays flat even for a 10 GB log.

5 Infinite streams, bounded on demand

A generator can run forever (`while True: yield`). That's safe precisely because it's lazy: you only ever pull what you take. Bound it with `islice` or `takewhile`.

infinite.py

```

from itertools import count, islice

def naturals():
    n = 0
    while True:
        yield n
        n += 1

list(islice(naturals(), 5)) # [0, 1, 2, 3, 4]
list(islice(count(10, 2), 3)) # 10, 12, 14 (the stdlib count)
  
```

WHY IT MATTERS

An eager list of "all the naturals" is impossible; a generator of them is one line. It's how you model event streams, retries, and paginated APIs: potentially endless, consumed one page at a time.

6 yield from — delegate to a sub-generator



`yield from` yields every value of another iterable (and passes its return value back). Since Python 3.3 (PEP 380) it replaces the manual re-yield loop.

```
yieldfrom.py

def chain(*iterables):
    for it in iterables:
        yield from it          # not: for x in it: yield x

list(chain([1, 2], (3, 4)))    # [1, 2, 3, 4]
```

7 itertools — the lazy toolkit

Before writing loops over a generator, check `itertools`: `islice`, `chain`, `count`, `groupby`, `takewhile`, `accumulate`. Two people often miss are newer:

```
itertools.py

from itertools import batched, pairwise

list(batched("ABCDEFGF", 3))  # [('A','B','C'), ('D','E','F'), ('G',)]
list(pairwise([1, 2, 3, 4])) # [(1,2), (2,3), (3,4)]
```

PRO TIP

`pairwise` landed in Python 3.10 and `batched` in 3.12 (with a `strict=` flag in 3.13). They're lazy and C-fast, so reach for them instead of hand-rolled index loops.



8 Common mistakes

- ▶ **Reusing an exhausted generator.** It yields nothing the second time. Keep a `list` if you truly need to iterate twice.
- ▶ `list(gen)` **on a huge stream.** That materialises everything and throws away the whole point; stay lazy end to end.
- ▶ `raise StopIteration` **to stop.** Since Python 3.7 (PEP 479) a `StopIteration` that escapes a generator is turned into a `RuntimeError`. Just `return`.
- ▶ **Expecting eager errors.** A bug inside the generator surfaces where you consume it, not where you define it.

9 Read more

Every snippet ships as a complete, dependency-free file in `codes/`. Primary sources:

- ▶ Python docs: the yield expression and the functional HOWTO.
- ▶ `itertools` reference: docs.python.org/3/library/itertools.
- ▶ PEP 380 (`yield from`) and PEP 479 (`StopIteration`).

Read the full guide

Eight runnable, dependency-free examples plus the complete write-up — clone the repo or read it right on GitHub.

